Python Code

```
"""
Map Search
"""

import comp140_module7 as maps

class Queue:
    """
    A simple implementation of a FIFO queue.
    """

    def __init__(self):
        self._queue = []

    def __len__(self):
        return len(self._queue)

    def __str__(self):
        return str(self._queue)

    def push(self, item):
        """
        Adds an item to the back of the queue.
        inputs:
            - item: the item to be added to the queue
        Returns: nothing
        """
        self._queue.append(item)

    def pop(self):
        """
        Removes and returns the item at the front of the queue.
        inputs: nothing
        Returns: the item at the front of the queue
        """
        if len(self._queue) == 0:
            raise IndexError("empty queue")
        return self._queue.pop(0)

    def clear(self):
        """
        Removes all items from the queue.
        inputs: nothing
        Returns: nothing
        """
        self._queue = []

class Stack:
    """
    A simple implementation of a LIFO stack.
```

```
    """
    def __init__(self):
        self._stack = []

    def __len__(self):
        return len(self._stack)

    def __str__(self):
        return str(self._stack)

    def push(self, item):
        """
        Adds an item to the top of the stack.
        inputs:
            - item: the item to be added to the stack
        Returns: nothing
        """
        self._stack.append(item)

    def pop(self):
        """
        Removes and returns the item at the top of the stack.
        inputs: nothing
        Returns: the item at the top of the stack
        """
        if len(self._stack) == 0:
            raise IndexError("empty stack")
        return self._stack.pop()

    def clear(self):
        """
        Removes all items from the stack.
        inputs: nothing
        Returns: nothing
        """
        self._stack = []


def bfs_dfs(graph, rac_class, start_node, end_node):
    """
    Performs a breadth-first search or a depth-first search on graph
    starting at the start_node.  The rac_class should either be a
    Queue class or a Stack class to select BFS or DFS.

    Completes when end_node is found or entire graph has been
    searched.

    inputs:
        - graph: a directed Graph object representing a street map
        - rac_class: a restricted access container (Queue or Stack) class to
          use for the search
        - start_node: a node in graph representing the start
        - end_node: a node in graph representing the end

```

```
    Returns: a dictionary associating each visited node with its parent
    node.
    """
    # initialize
    dist = {}
    parent = {}

    for node in graph.nodes():
        dist[node] = float('inf')
        parent[node] = None

    #  start
    dist[start_node] = 0

    # RAC:queue or stack
    rac = rac_class()
    rac.push(start_node)

    # iterate over graph
    while len(rac) > 0:
        node = rac.pop()

        # stop early if end is found
        if node == end_node:
            return parent

        # iterate over neighbors
        for nbr in graph.get_neighbors(node):
            if dist[nbr] == float('inf'):
                dist[nbr] = dist[node] + 1
                parent[nbr] = node
                rac.push(nbr)

    return parent

def dfs(graph, start_node, end_node, parent):
    """
    Performs a recursive depth-first search on graph starting at the
    start_node.

    Completes when end_node is found or entire graph has been
    searched.

    inputs:
        - graph: a directed Graph object representing a street map
        - start_node: a node in graph representing the start
        - end_node: a node in graph representing the end
        - parent: a dictionary that initially has one entry associating
                  the original start_node with None

    Modifies the input parent dictionary to associate each visited node
    with its parent node
    """
    if start_node == end_node:
```

```
        return

    # iterate over neighbors
    for nbr in graph.get_neighbors(start_node):
        #unvisited
        if nbr not in parent:
            # update parent
            parent[nbr] = start_node
            # recrusion
            dfs(graph, nbr, end_node, parent)

            # end node found early
            if end_node in parent:
                return

    return

def astar(graph, start_node, end_node,
          edge_distance, straight_line_distance):
    """
    Performs an A* search on graph starting at start_node.

    Completes when end_node is found or entire graph has been
    searched.

    inputs:
        - graph: a directed Graph object representing a street map
        - start_node: a node in graph representing the start
        - end_node: a node in graph representing the end
        - edge_distance: a function which takes two nodes and a graph
                         and returns the actual distance between two
                         neighboring nodes
        - straight_line_distance: a function which takes two nodes and
                         a graph and returns the straight line distance
                         between two nodes

    Returns: a dictionary associating each visited node with its parent
    node.
    """
    #initialize
    open_set = set([start_node])
    parent = {start_node: None}

    #  distance from start_node to each node
    g_value = {start_node: 0}

    # heuristic + distance
    f_value = {
        start_node: straight_line_distance(start_node, end_node, graph)
    }

    while len(open_set) > 0:
        # find minimm  f val
        current = min(open_set, key=lambda node: f_value[node])
```

```
213
214         if current == end_node:
215             return parent
216
217         open_set.remove(current)
218
219         # iterate
220         for nbr in graph.get_neighbors(current):
221             tent_g = g_value[current] + edge_distance(current, nbr, graph)
222
223             # update g and f values
224             if nbr not in g_value or tent_g < g_value[nbr]:
225                 g_value[nbr] = tent_g
226                 f_value[nbr] = tent_g + straight_line_distance(nbr, end_node, graph)
227                 parent[nbr] = current
228                 open_set.add(nbr)
229
230     return parent
231
232 # You can replace functions/classes you have not yet implemented with
233 # None in the call to "maps.start" below and the other elements will
234 # work.
235
236 maps.start(bfs_dfs, Queue, Stack, dfs, astar)
237
```